

i) What is the role of optimizers?

It's primary role is to minimize the model's error or loss function, enhancing Performance.

ii) Calculate updates for model Parameters

iii) Minimize the loss function

(Reduce the Prediction error improve model accuracy)

iv) Control learning speed

v) Improves Convergence.

(Optimal solution faster more accuracy)

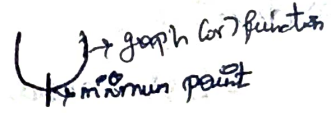
vi) Avoid the problem like local minima and oscillation.

a) TYPES of gradient Descent approach:

1) Batch gradient Descent approach:

gradients using the entire dataset in each iteration.

Advantage:

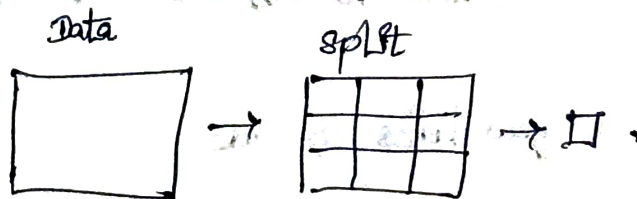
- Steady and accurate convergence to a minimum
- Easy Implementation
- Excellent for Convex function 

Disadvantage:

- unpractical for large dataset.
- Slow to converge
- Difficult to Escape Local minima.

2) mini-batch gradient Descent:

Combines Batch and SGD by using small batches of data for updates



— large to split in minimum.

Advantage:

- Provide greater accuracy than stochastic gradient Descent.
- Able to handle large Dataset.
- " " escape Local minima.

Disadvantage:

- more complex Algorithm
- Additional Hyperparameter
- less accurate than batch gradient Descent.

2) Stochastic gradient Descent: (Use for RMS Prop)
Uses on data point Per iteration to compute gradients making it faster.

Advantage:

- Faster Convergence - suitable for large Dataset.
- Able to escape local minima
- no Additional Hyperparameter

Disadvantage:

- less accurate Convergence
- more complex implementation
- non-deterministic.

3) Types of optimizer (Advantage & disadvantage):

✓ 1) Gradient Descent:

AD:

→ GD is a very efficient algorithm for optimizing model with a large number of Parameters.

→ Algorithm can be applied to different type of models.

→ Requires only a few lines of code.

DA:

→ local minima.

→ Requires tuning (Finding optimal hyperparameters)

Can be time consuming and require a lot of trial and error.

→ Sensitive feature scaling.

2) Stochastic:

Ad:

→ Faster Convergence

→ Lower memory requirements

→ Ability to handle noisy data.

→ Generalization

Disadvantage:

- Possibility of getting stuck in local minima.
- Need for careful tuning.
- Sensitivity to initialization.
- May require more iteration.

3. Adam:

Ad:

- Combines the benefits of momentum and Adagrad.
- Adapts the learning rate for each parameter based on both the first and second moments of their past gradients.
- works well for a wide range of problems and architectures

DA:

- may suffer from overfitting when used with smaller dataset.

4. Adagrad:

Ad:

- Each Parameter based on the sum of the squares of its past gradient, suitable for sparse dataset.
- Perform well for low learning Rate.

DA:

- learning rate become too smaller over time.
- Require more memory than SGD due to the need to store past gradients sum.

5. RMS Prop:

Ad: → steps to control

- Perform well various domain. → Handle horizontal.
- Non-stationary to control. → Handle different learning rate.

D.A:

- may Converge slowly Compared to other optimizers
- not suitable for Problems with sparse data

6. Adadelta:

AD:

- Address the small learning rate problem by using an exponentially decaying average of Past gradient.
- require less memory. only need to store Past gradient.

D.A:

- learning rate can become too small to overcome overfitting
- many Converge more slowly than other optimizers.

1) Regularisation: (Model reduce training error add againe Pathology)

Type:

L_1 - Lasso - All weights to 0.

L_2 - Ridge, - large weight to reduce.

Ad:

- Reduces overfit
- prevent model from becoming too complex.
- Improve generalisation of unseen data.

DA:

- Choosing the regularization parameter (this difficult)
- may increase training time slightly.
- too much regularization Cause overfitting

2) Dropout:

Training time random neurones to be disabled.

Ad:

- Improve model generalisation
- prevents neurones from depending too much on each other
- simple & easy to implement.

DA:

- Training become slower.
- choosing dropout rate is tricky
- Too much dropout can reduce model accuracy.

3) Early Stopping

Ad:

- prevent over-training
- Reduce overfitting

→ Save training time and Computational Cost

D.A :

→ Requires Validation dataset

→ Choosing patience / stop criteria is difficult

→ may stop too early before optimal learning.